KENNESAW STATE
UNIVERSITY

CS 4277
Linear Regression

Dr. Chen Zhao

czhao4@kennesaw.edu

Computer Science

College of Computing and Software Engineering

Learning Objective

- Auto differentiation (AD)
- Supervised Learning
- Univariate Linear Model
- Optimization
- Multivariate Linear Regression
- Example

Auto Differentiation

Auto Differentiation (AD)

- AD evaluates gradients of a function specified by a program at given values
- AD differs to
  - Symbolic differentiation

```
In[1]:= D[4 x^3 + x^2 + 3, x]

Out[1]= 2 x + 12 x^2
```

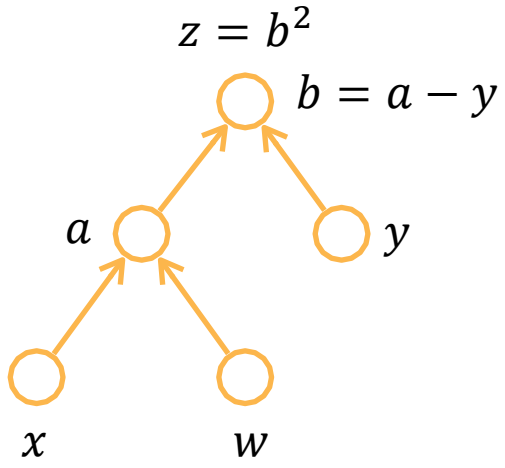
- Numerical differentiation (Finite difference)

$$\frac{\partial f(x)}{\partial x} = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

Computation graph

- Decompose into primitive operations
- Build a directed acyclic graph to present the computation
- Build *explicitly*
  - TensorFlow1/Theano/MXNet
- Build *implicitly* though tracing
  - PyTorch/TensorFlow2/MXNet

Assume $z = (\langle \boldsymbol{x}, \boldsymbol{w} \rangle - y)^2$



Two Modes

- By chain rule

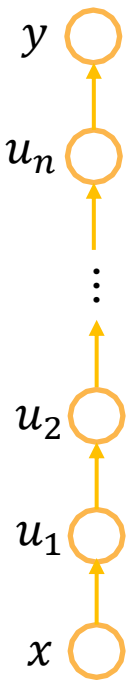
$$\frac{\partial y}{\partial x} = \frac{\partial y}{\partial u_n}\frac{\partial u_n}{\partial u_{n-1}}\cdots\frac{\partial u_2}{\partial u_1}\frac{\partial u_1}{\partial x}$$

- Forward accumulation

$$\frac{\partial y}{\partial x} = \frac{\partial y}{\partial u_n}\left(\frac{\partial u_n}{\partial u_{n-1}}\left(\cdots\frac{\partial u_2}{\partial u_1}\frac{\partial u_1}{\partial x}\right)\right)$$

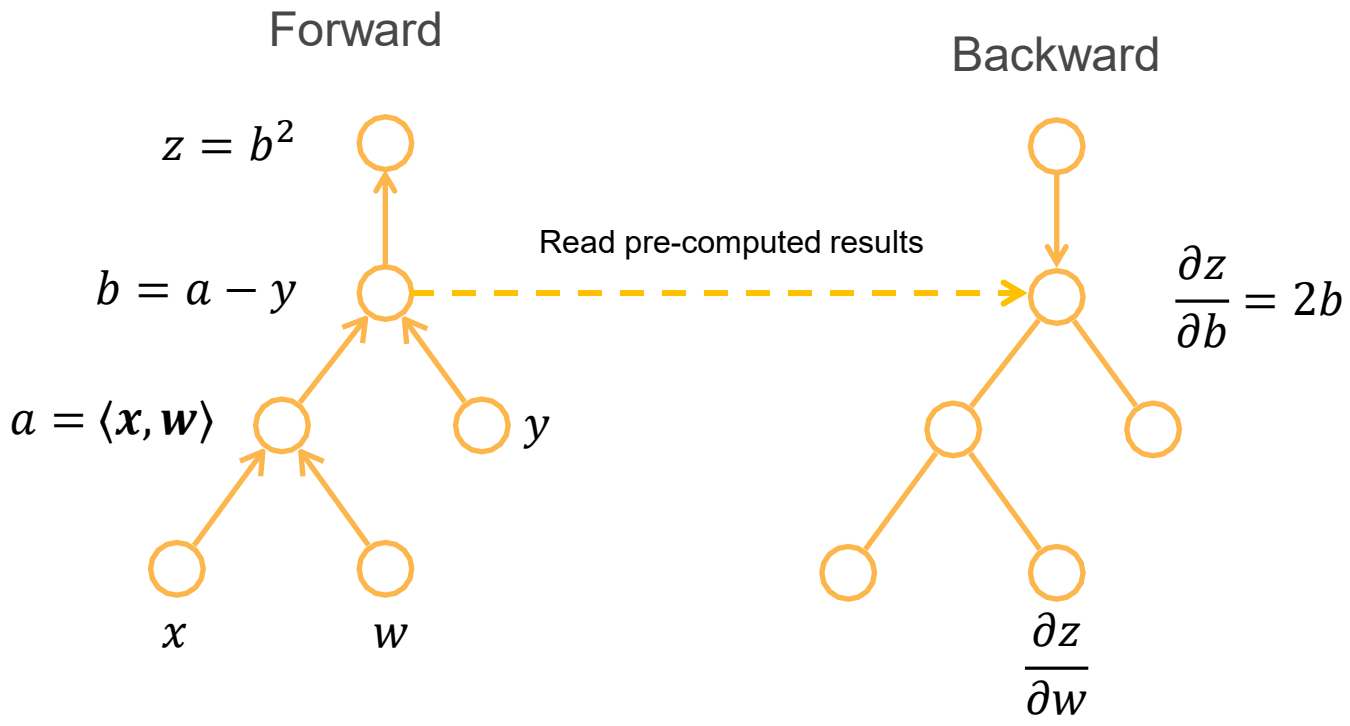
- Reverse accumulation (a.k.a Backpropagation)

$$\frac{\partial y}{\partial x} = \left(\left(\frac{\partial y}{\partial u_n}\frac{\partial u_n}{\partial u_{n-1}}\right)\cdots\frac{\partial u_2}{\partial u_1}\right)\frac{\partial u_1}{\partial x}$$



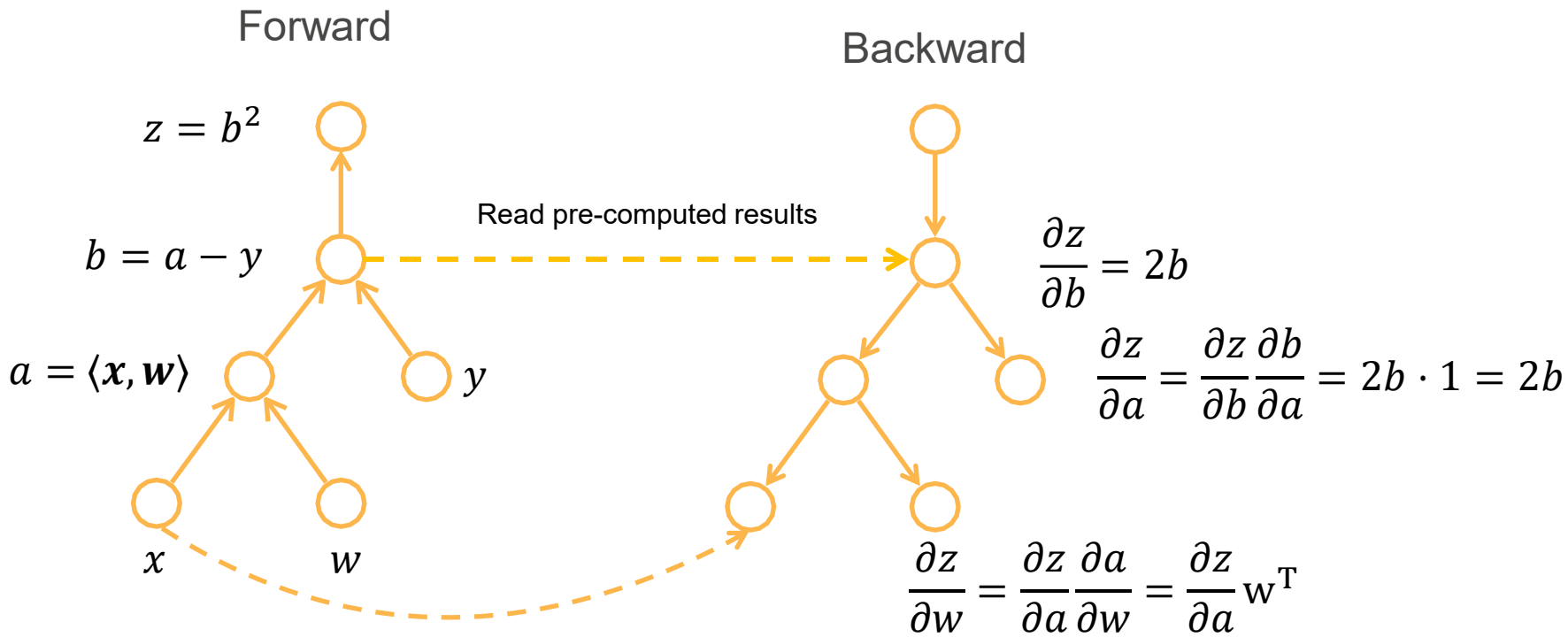
Reverse Accumulation

Assume $z = (\langle \boldsymbol{x}, \boldsymbol{w} \rangle - y)^2$



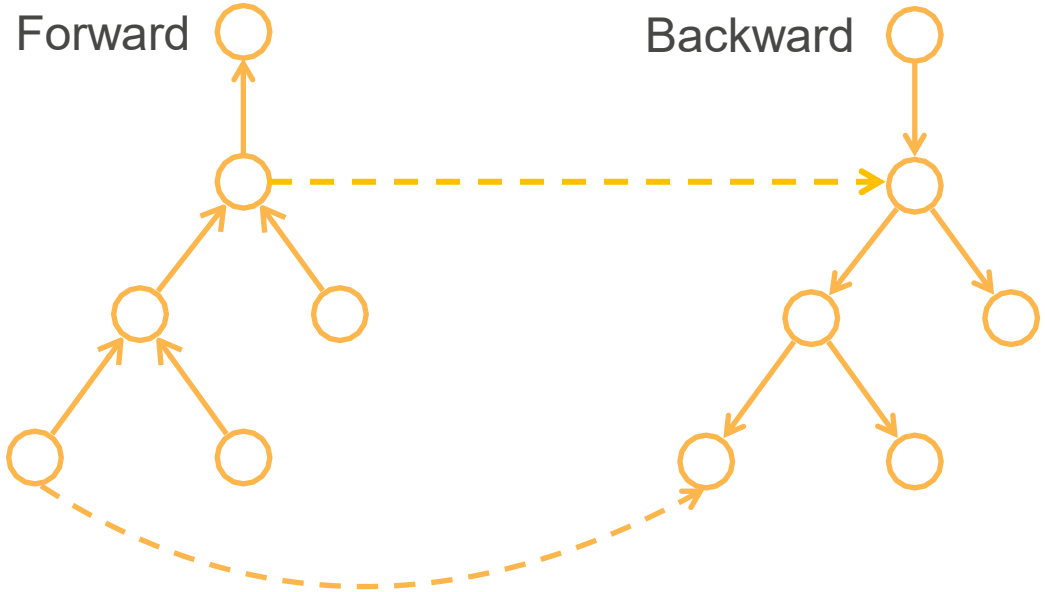
Reverse Accumulation

Assume $z = (\langle \boldsymbol{x}, \boldsymbol{w} \rangle - y)^2$



Reverse Accumulation Summary

- Build a computation graph
- Forward: Evaluate the graph, store intermediate results
- Backward: Evaluate the graph in a reversed order
  - Eliminate paths not needed



Supervised Learning

Recap: supervised learning

- Input: x (Images, texts, emails)
- Output: y (e.g., digit, cat or dog, spam or non-spam)

- Data: $\left(x^{(1)}, y^{(1)}\right), \left(x^{(2)}, y^{(2)}\right), \cdots, \left(x^{(N)}, y^{(N)}\right)$(Labeled dataset)
- (Unknown) Target function: $f : x \rightarrow y$ ("True" mapping)

- Model/hypothesis: h: $x \rightarrow y$ (Learned model)

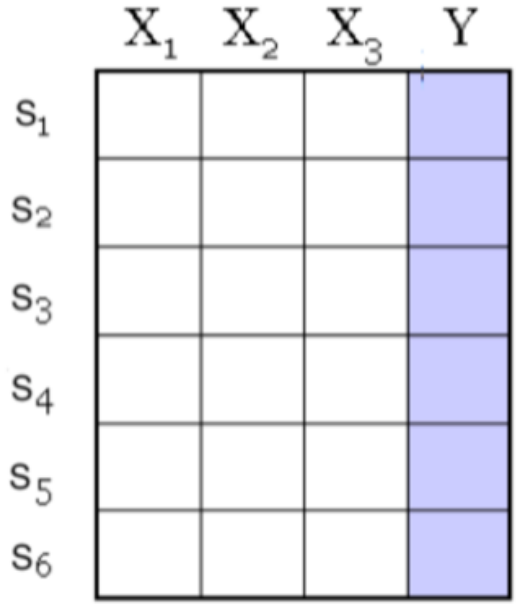
- Learning = search in hypothesis space

Recap: Workflow of supervised learning

- Set up a supervised learning problem
- Data collection: Collect training data with the “right” answer.
- Representation: Choose how to represent the data.
- Modeling: Choose a hypothesis class: $H = \{ h: X \rightarrow Y \}$
- Learning/estimation: Find best hypothesis in the model class.
- Model selection: Try different models. Picks the best one. (More on this later)
- If happy stop, else refine one or more of the above

Terminology

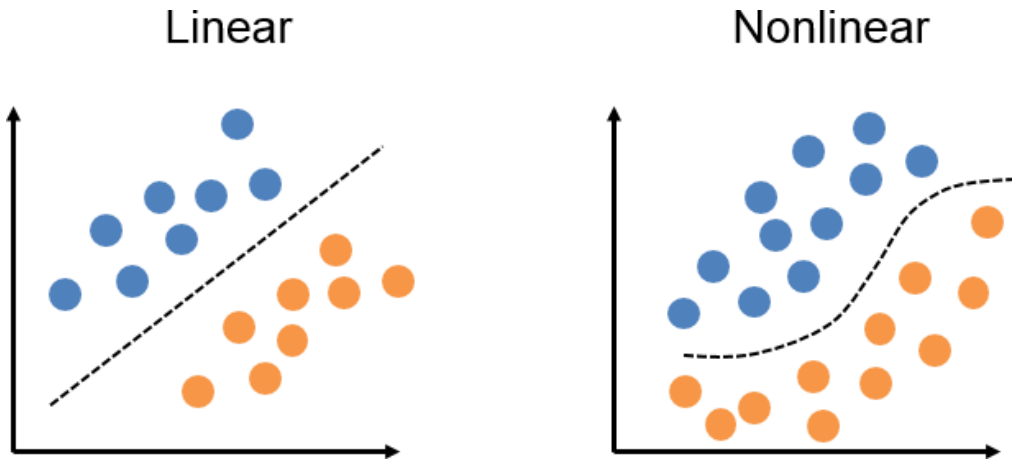
- Data/points/instances/examples/samples/records: [rows]
- Features/attributes/dimensions/independent variables/covariates/predictors/regressors: [columns, except the last]
- Label/Target/outcome/response/dependent variable: special column to be predicted [last column]

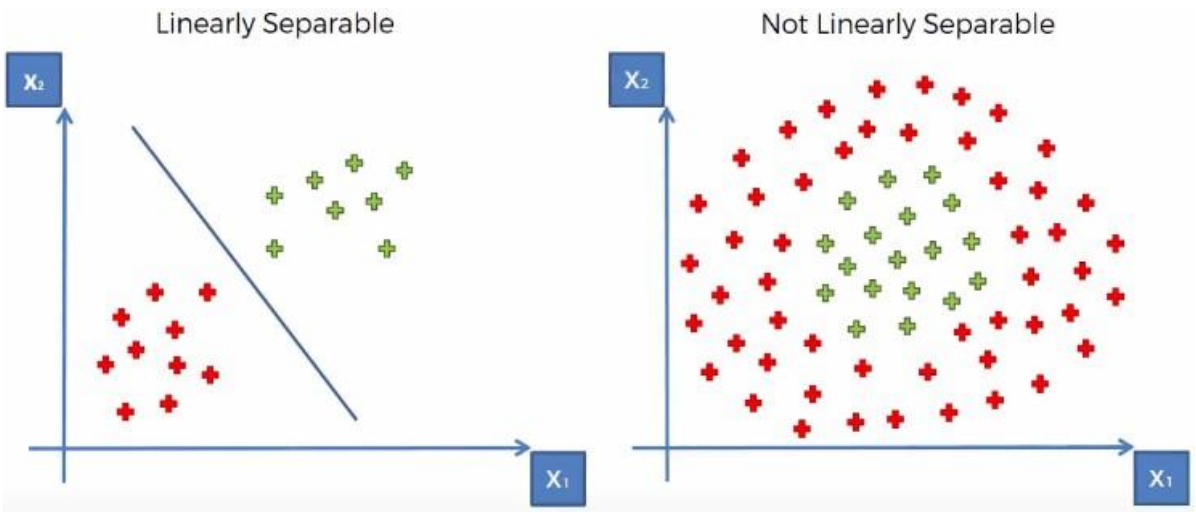


$$f : x \rightarrow y$$

Linear Model

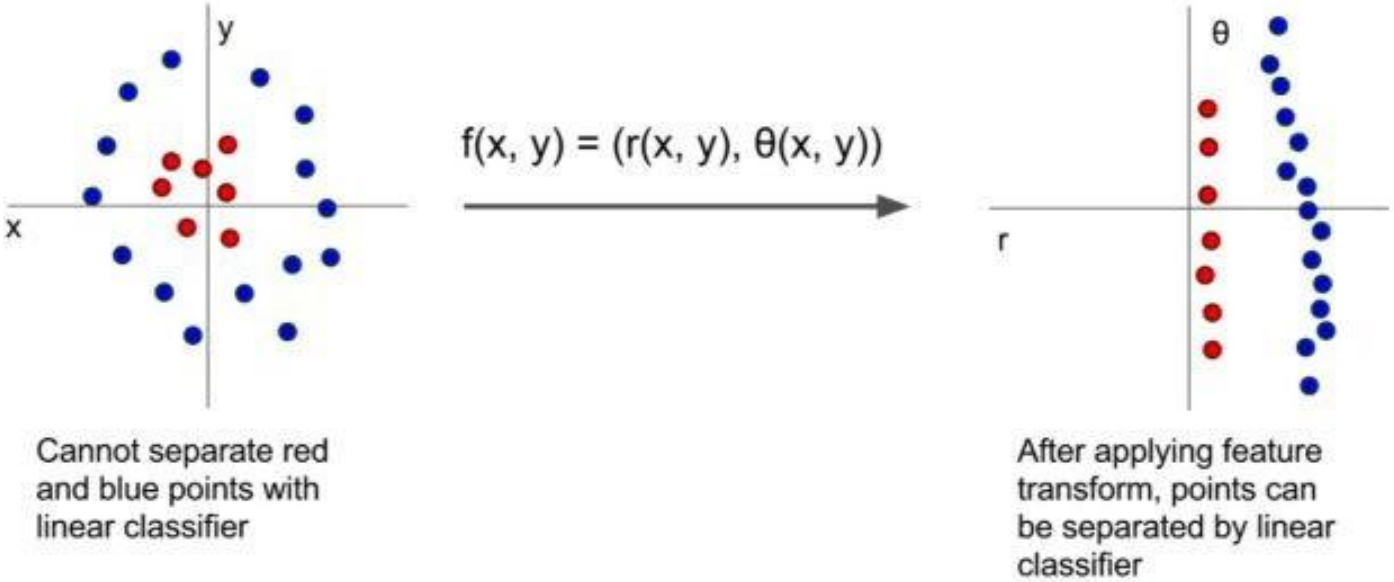
Linear and non-linear classifiers





How to use a linear classifier to classify a set of linearly inseparable data?

- First grab some features of the group of data (higher dimensional space)
- Then classify these features linearly

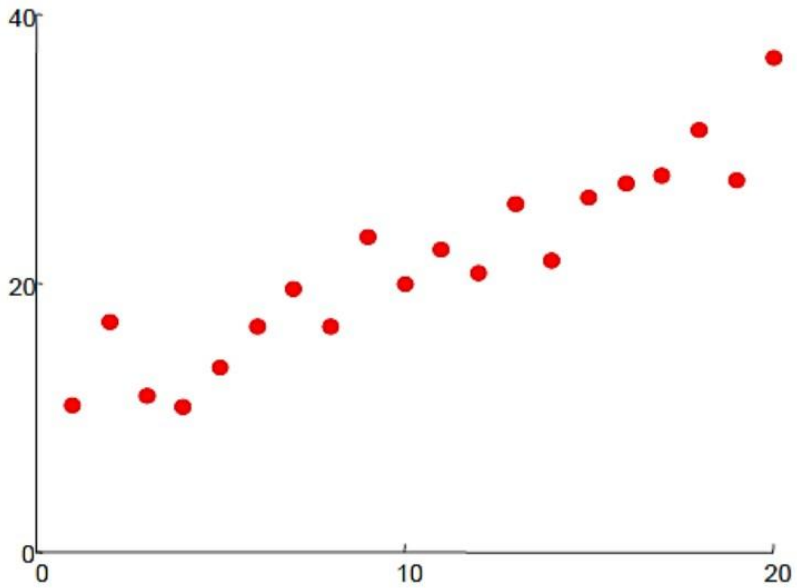


Linear Regression

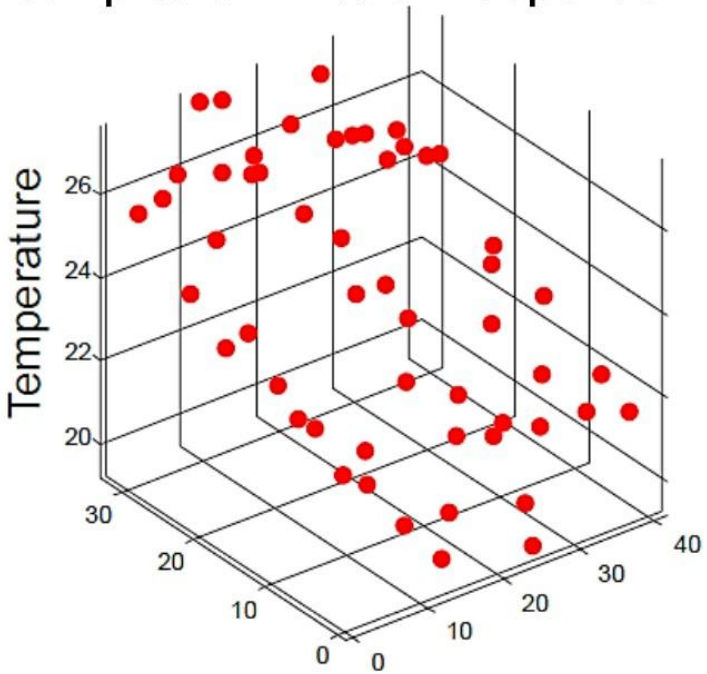
- First look at examples that help us understand intuitively:
- Give you the following X and Y values, (1,1), (2,2), (4,4), (100,100), (20, 20), when $X = 5$, what is Y?
  - $Y = 5$
- First, establish the relationship between X and Y through the given data, that is $Y = X$
- Then, using the established relationship, predict the unknown Y value by the given X value

Linear Regression

Samples with ONE independent variable



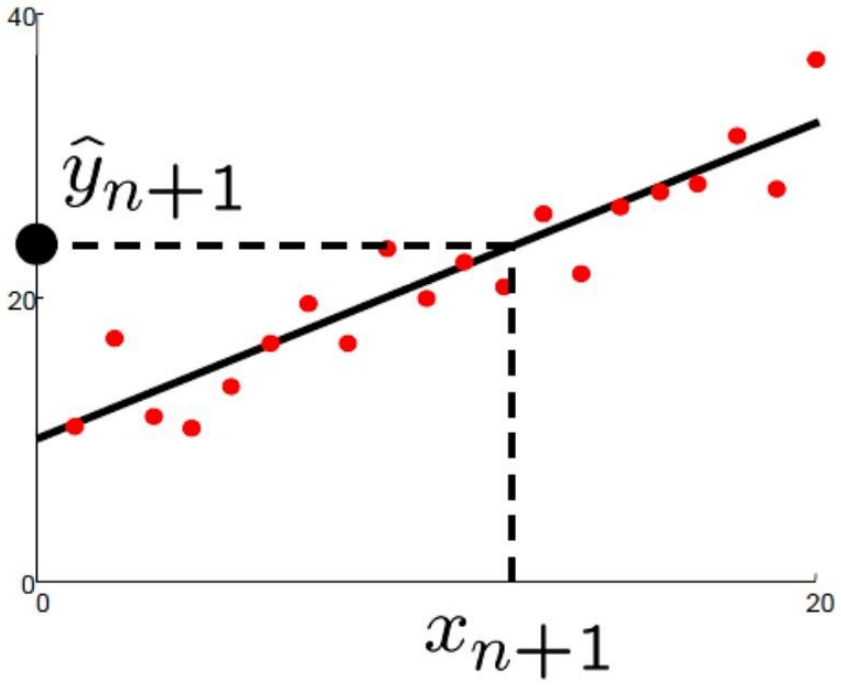
Samples with TWO independent variables



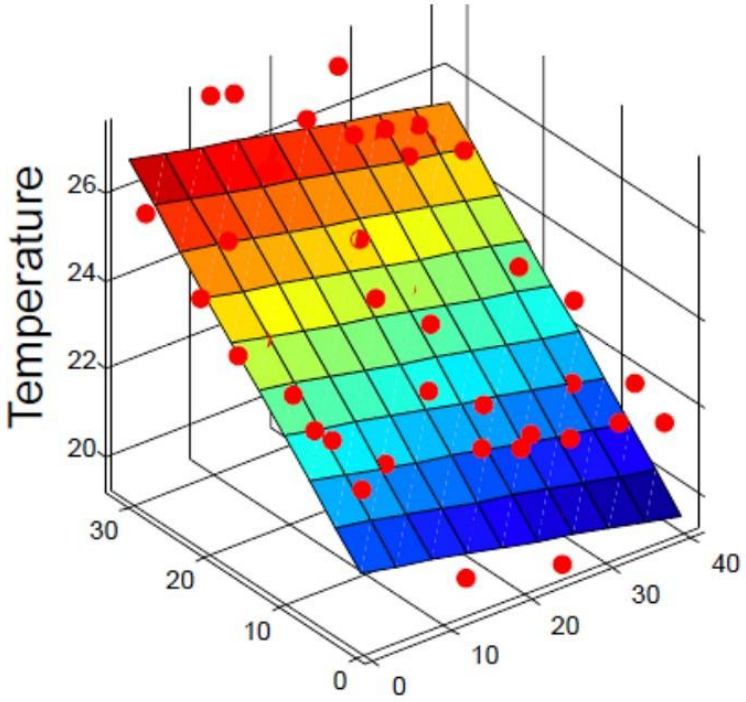
- Given examples $(x_i, y_i)\ s.t.\ i \in \{1, \cdots n\}$
- Predict y_{n+1} given a new point x_{n+1}

Linear Regression

Samples with ONE independent variable



Samples with TWO independent variables



Linear Regression

- Simple linear regression
  - A single independent variable is used to predict

$$y_i = \beta_0 + \beta_1 x_i$$

- $i \in \{1, \cdots, n\}$: n observations
- x_i: independent variables
- y_i: dependent variables
- β_0: intercept (constant term)
- β_p: slope coefficients for each dependent variables
- ϵ: model's error term (residuals)

- Multiple linear regression
  - Two or more independent variables are used to predict

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip} + \epsilon$$

Basic elements in sample linear regression (single/uni variable)

$$y_i = \beta_0 + \beta_1 x_i$$

- x_i represents the input variable (independent variable)
- y_i represents the output variable (dependent variable)
- A pair of (x_i, y_i) represents a training sample
- i represents the ith training sample
- n training samples (x_i, y_i), s.t. $i \in \{1, \cdots, n\}$ represents training set
- $X = \{x_1, x_2, \cdots, x_n\}$:the space composed of all input values/features
- $Y = \{y_1, y_2, \cdots, y_n\}$:the space composed of all output values/labels

Linear regression

- Given a training set, our goal is to "learn" to get a function h: $X -> Y$, so that $h(x)$ is a "good" predicted value of the true value y

- $\theta = (\theta_0, \theta_1)$ represent the parameters of the model

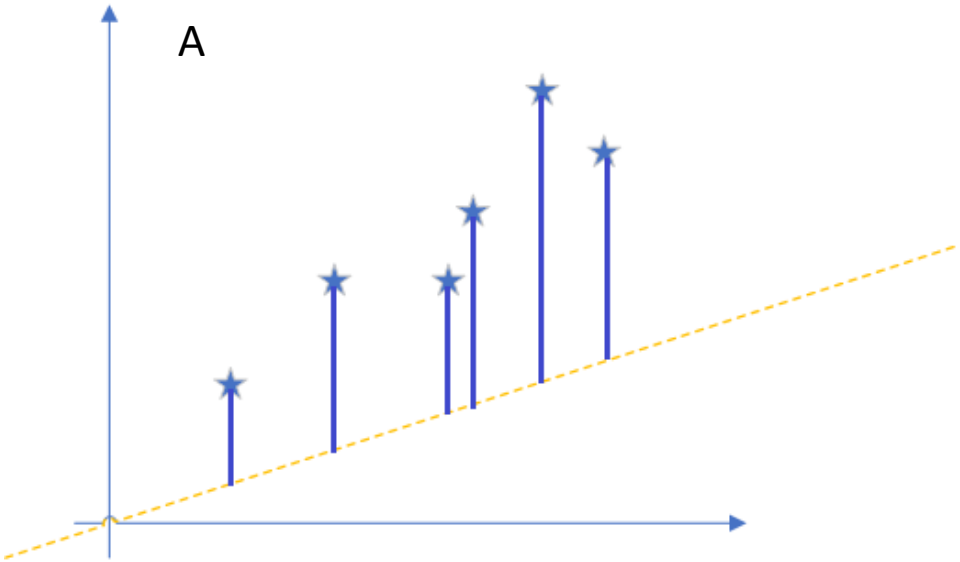
- The goal of linear regression is to find the most suitable θ_0 and θ_1 to make the model effect the best

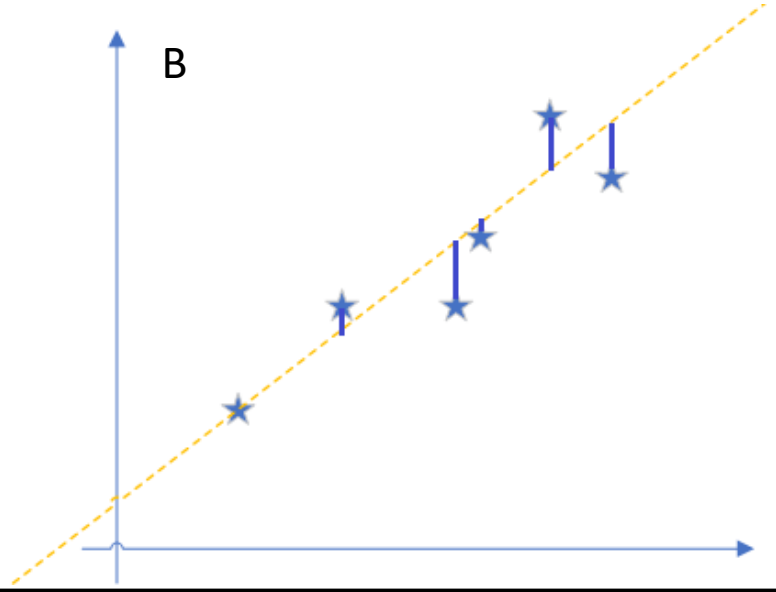
Linear regression

- Assuming there are some points like this, we want to find a line that fits these points, and then predict the next points
- $y(x) = \theta_0 + \theta_1 x$
- How to find θ_0 and θ_1?

Cost Function

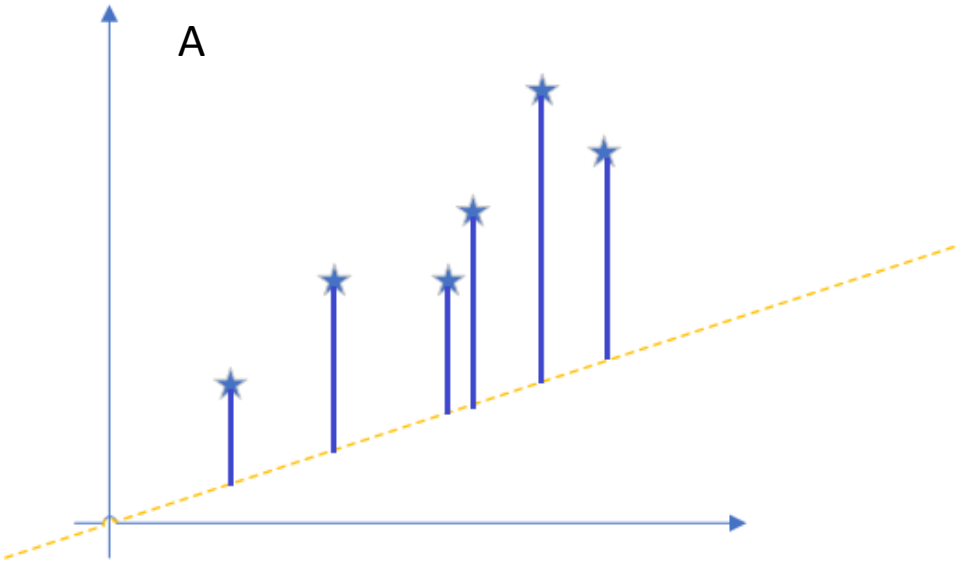
- Least square error: $J = \frac{1}{n}\sum_{i=1}^{n}(pred_i - y_i)^2$
- B is more fitting, which means that the line in B is closer to the ideal line

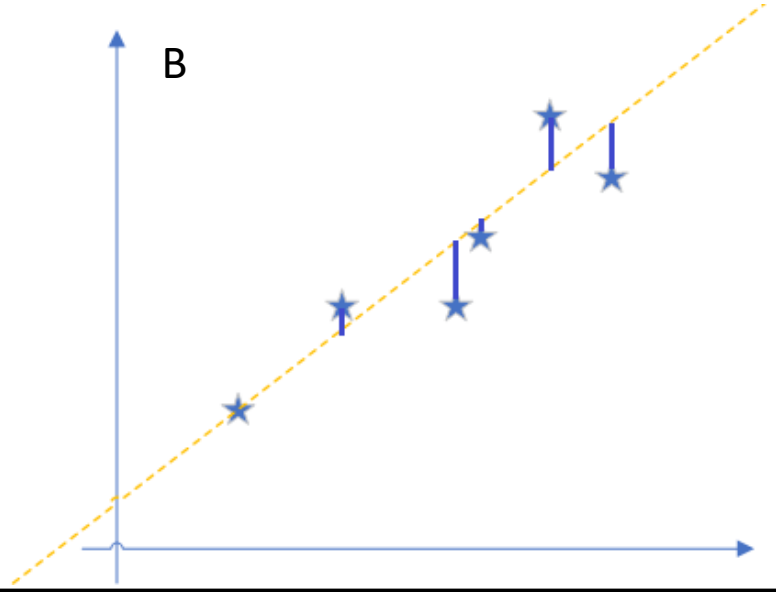




Cost Function

- Aim: minimize $J = \frac{1}{n}\sum_{i=1}^{n}(pred_i - y_i)^2$
- $pred_i = \theta_0 + \theta_1 x_i$, thus, the optimal θ_0 and θ_1 are derived
- B is more fitting, which means that the line in B is closer to the ideal line





Optimization

- $J = \frac{1}{n}\sum_{i=1}^{n}(pred_i - y_i)^2 = \frac{1}{n}\sum_{i=1}^{n}(\theta_0 + \theta_1 x_i - y_i)^2$
- Optimization:
  - Take the derivate with respective to (w.r.t.) θ_1
  - $\frac{\partial J}{\partial \theta_1} = \frac{2}{n}\sum_{i=1}^{n}(\theta_0 + \theta_1 x_i - y_i) \cdot x_i = \frac{2}{n}\sum_{i=1}^{n}(pred_i - y_i) \cdot x_i$

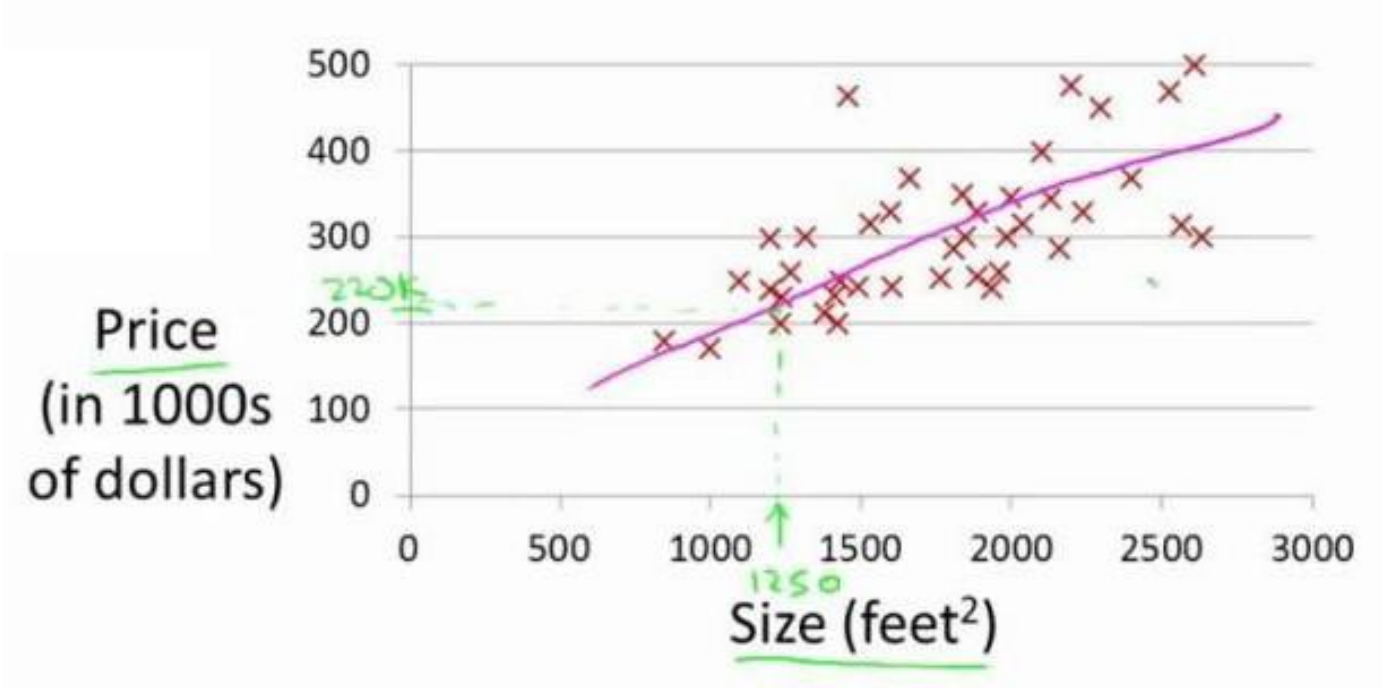
  - Take the derivate with respective to (w.r.t.) θ_0
  - $\frac{\partial J}{\partial \theta_0} = \frac{2}{n}\sum_{i=1}^{n}(\theta_0 + \theta_1 x_i - y_i) = \frac{2}{n}\sum_{i=1}^{n}(pred_i - y_i)$

Optional: solution

- $\begin{cases} \frac{\partial J}{\partial \theta_1} = \frac{2}{n}\sum_{i=1}^{n}(\theta_0 + \theta_1 x_i - y_i)\cdot x_i = 0 \\ \frac{\partial J}{\partial \theta_0} = \frac{2}{n}\sum_{i=1}^{n}(\theta_0 + \theta_1 x_i - y_i) = 0 \end{cases}$
- $\frac{\partial J}{\partial \theta_0} = \frac{2}{n}\sum_{i=1}^{n}(\theta_0 + \theta_1 x_i - y_i) = 0 \Longrightarrow n\theta_0 + \sum_{i=1}^{n}(\theta_1 x_i - y_i) = 0 \Longrightarrow \theta_0 = \frac{1}{n}\cdot\sum_{i=1}^{n}(y_i - \theta_1 x_i)$
- $\frac{\partial J}{\partial \theta_1} = 0 \Longrightarrow \sum_{i=1}^{n}\left((\theta_0 + \theta_1 x_i - y_i)\cdot x_i\right) = 0 \Longrightarrow \theta_0 \sum_{i=1}^{n} x_i + \theta_1 \sum_{i}^{n} x_i^2 - \sum_{i}^{n} x_i y_i = 0$
- Substitute θ_0 to $\frac{\partial J}{\partial \theta_1} = 0 \Longrightarrow \frac{1}{n}\cdot\sum_{j=1}^{n}(y_j - \theta_1 x_j)\sum_{i=1}^{n} x_i + \theta_1 \sum_{i=1}^{n} x_i^2 - \sum_{i=1}^{n} x_i y_i = 0$
- $\frac{1}{n}\sum_{j=1}^{n} y_j \sum_{i=1}^{n} x_i - \frac{\theta_1}{n}\sum_{i=1}^{n} x_i \sum_{j=1}^{n} x_j + \theta_1 \sum_{i=1}^{n} x_i^2 - \sum_{i=1}^{n} x_i y_i = 0$
- $\left[\sum_{i=1}^{n} x_i^2 - \frac{1}{n}\left(\sum_{i=1}^{n} x_i\right)^2\right]\theta_1 = \sum_{i=1}^{n} x_i y_i - \frac{1}{n}\sum_{j=1}^{n} y_j \sum_{i=1}^{n} x_i$
- $\theta_1 = \frac{\sum_{i=1}^{n} x_i y_i - \frac{1}{n}\sum_{j=1}^{n} y_j \sum_{i=1}^{n} x_i}{\sum_{i=1}^{n} x_i^2 - \frac{1}{n}\left(\sum_{i=1}^{n} x_i\right)^2}$

Example: predict house price

Size in feet^2 (x)	Price ($) in 1000's (y)
2104	460
1416	232
1534	315
852	178
…	…



- If your friend's house is 1,250 square feet in size, tell them how much the house can sell

Example: predict house price

- Hypothesis: $h_\theta(x) = \theta_0 + \theta_1 x$
- Parameters: θ_0, θ_1
- Cost function: Least square cost: $J = \frac{1}{n}\sum_{i=1}^{n}(h_\theta(x_i) - y_i)^2$
- Goal: $\min_{\theta_0,\theta_1} J$

Solve Linear Regression

Two Methods to Solve

- Normal Equation
- Gradient Descent

Normal Equation

- Given a full rank matrices $X \in \mathbb{R}^{n \times m}$ (n samples with m features), and a vector $y \in \mathbb{R}^n$, we want to find a vector θ that $X\theta$ is as close as possible to y, as measured by the square of Euclidean norm $\|X\theta - y\|_2^2$.
- Using the fact that $\|\theta\|_2^2 = \theta^T\theta$, we have

$$\|X\theta - y\|_2^2 = (X\theta - y)^T(X\theta - y) = \theta^T X^T X\theta - 2y^T X\theta + y^T y$$

- Taking the gradient with respect to θ, we have

$$\nabla_\theta(\theta^T X^T X\theta - 2y^T X\theta + y^T y) = 2X^T X\theta - 2X^T y$$

- Setting this expression equal to zero and solving for θ gives the normal equations:

$$\theta = (X^T X)^{-1} X^T y$$

Example of Normal Equation

- Find a straight line to fit the points (1,1), (1,2), (1,3) and $y = (1,2,2)^T$ suppose $h_\theta(x) = \theta_0 + \theta_1 x$

- Mathematically, $X = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix}$ and $y = \begin{bmatrix} 1 \\ 2 \\ 2 \end{bmatrix}$

- $\theta = (X^T X)^{-1} X^T y = \begin{bmatrix} \frac{2}{3} \\ \frac{1}{2} \end{bmatrix}$

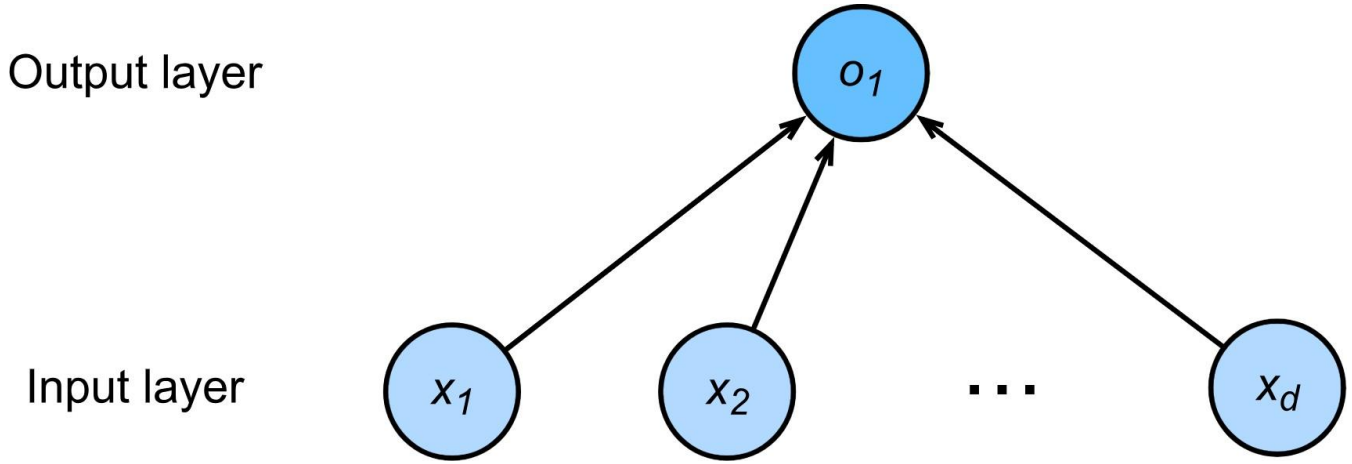
```
>>> import numpy as np
>>> X = np.array([[1, 1], [1,2], [1,3]])
>>> y = np.array([1,2,2]).T
>>> np.dot(np.linalg.pinv(X), y)
array([0.66666667, 0.5       ])
```

Pros/cons of Normal Equation

- Advantage of Normal Equation
  - Can be solved in one time
- Disadvantage of Normal Equation
  - Large amount of calculation. Especially when the dimension of the input variable is large (m is relatively large), the computational complexity of the algorithm increases exponentially.
  - Take up resources. Since the results need to be calculated in the memory at one time, when the amount of data is relatively large, the algorithm requires a lot of computer resources.

Linear Model as a Single-layer Neural Network

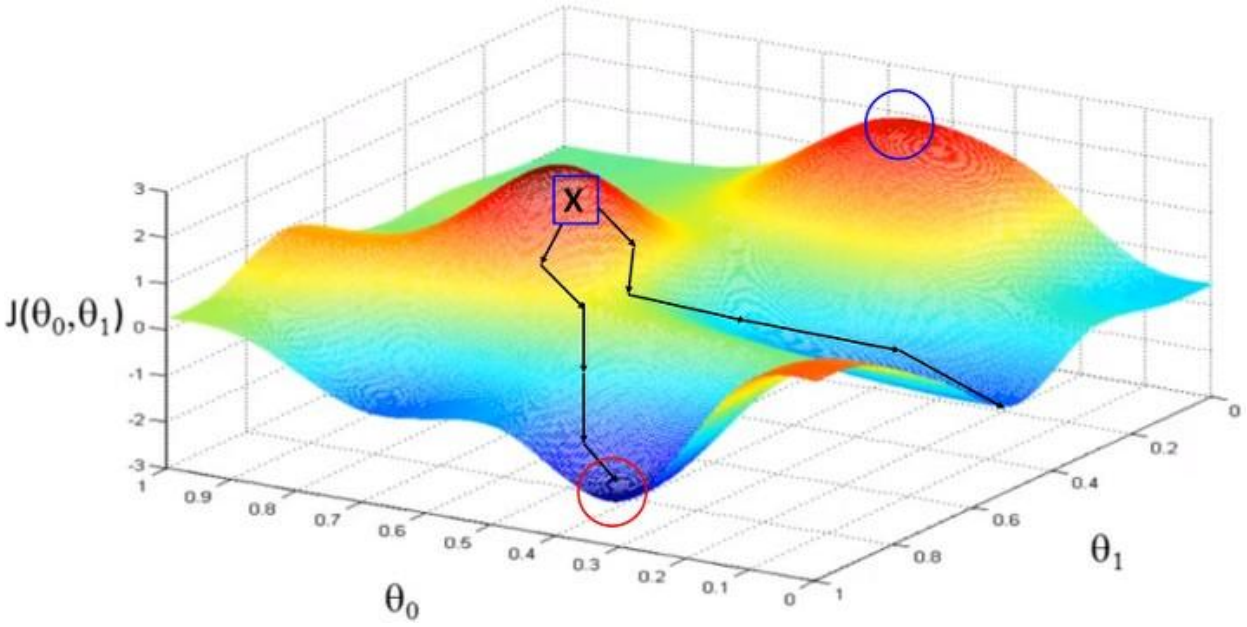
- We can stack multiple layers to get deep neural networks



Gradient Descent

Gradient descent

- Gradient descent is another method of solving parameters θ
- The blue area in the figure is the point with the smallest cost function. How to find the corresponding lowest point θ?



Gradient Descent

- How to find the corresponding lowest point θ ?
- The answer is to take the partial derivative of the cost function:

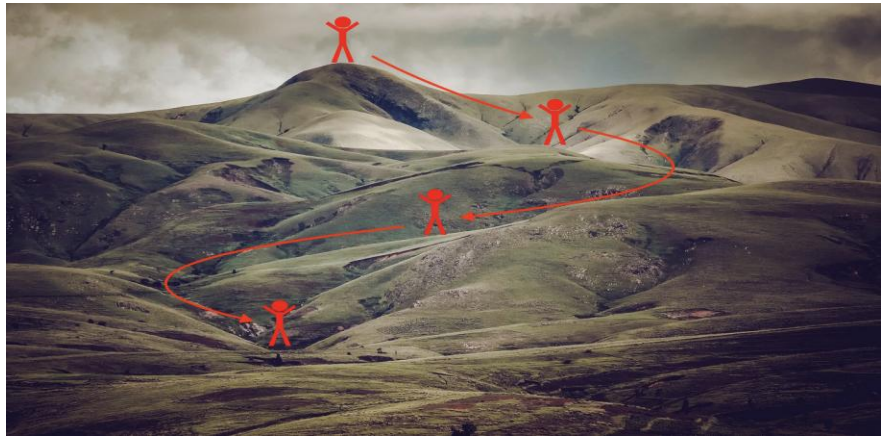
$$\frac{\partial J}{\partial \theta_j} J(\theta) = \frac{\partial J}{\partial \theta_j} \frac{1}{2} (h_\theta(x) - y)^2 = \frac{1}{2} \times 2 \times (h_\theta(x) - y) \frac{\partial J}{\partial \theta_j} \left(\sum_i^n \theta_i x_i - y \right) = (h_\theta(x) - y) x_j$$

- The meaning of finding the partial derivative is to get the slope of the tangent at this point, which will give us a direction to move towards the minimum. Keep moving until convergence.

$$\theta_j := \theta_j - \alpha \frac{\partial J}{\partial \theta_j} J(\theta)$$

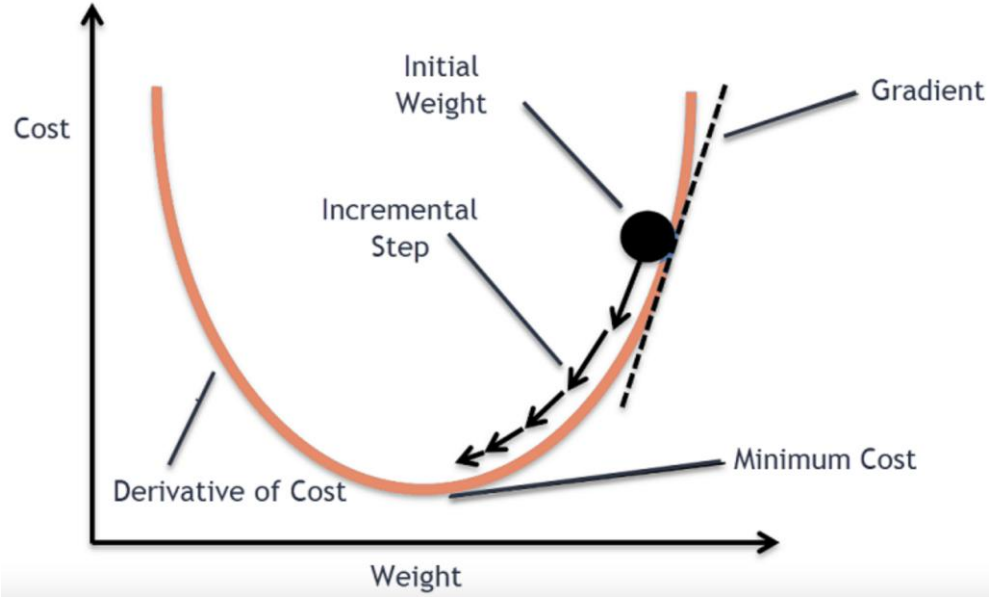
Intuitive gradient descent

- Cost function quantifies the error between predicted values and expected values
- Assume
  - You are playing a game where the players are at the top of a mountain, and they are asked to reach the lowest point of the mountain. Additionally, they are blindfolded. So, what approach do you think would make you reach the lake?
  - best way is to observe the ground and find where the land descends.
  - From that position, take a step in the descending direction and iterate this process until we reach the lowest point.
  - Gradient descent is an iterative optimization algorithm for finding the local minimum of a function.



Gradient descent

- Gradient descent is an iterative optimization algorithm for finding the local minimum of a function.
- local minimum of a function using gradient descent
  - Gradient Descent: take steps proportional to the negative of the gradient (move away from the gradient) of the function at the current point – Local Minimum
  - Gradient Ascent: take steps proportional to the positive of the gradient (moving towards the gradient) – Local Maximum

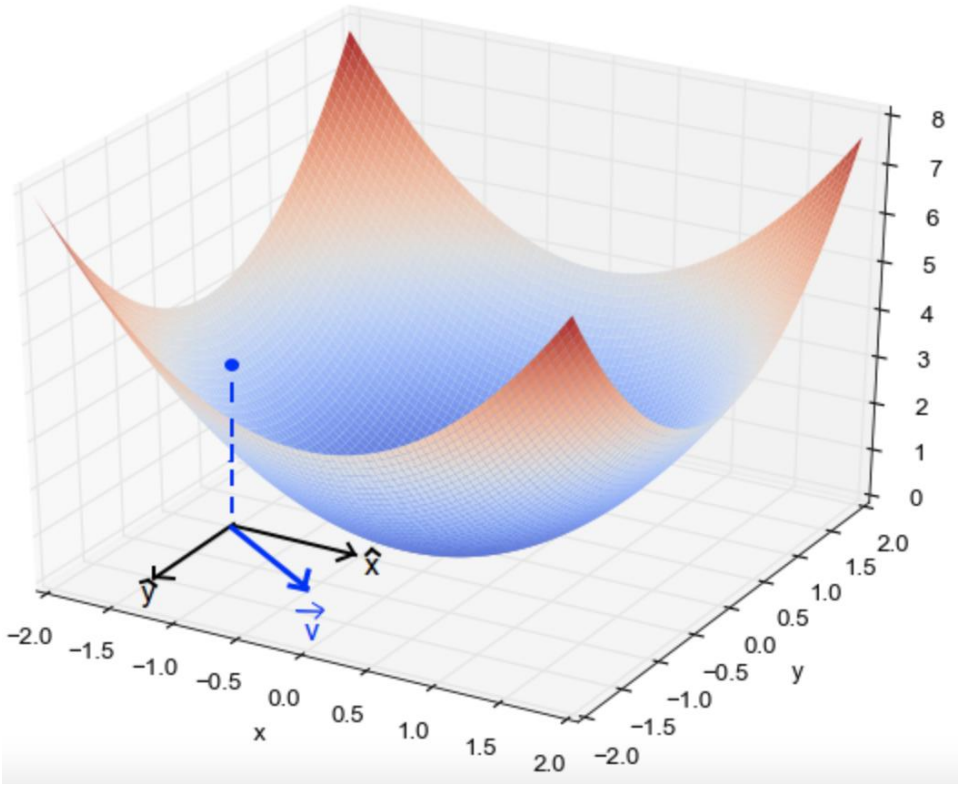


Two Steps to minimize cost function:

1. **Compute the gradient** (slope)
2. **Make a step (move) in the direction opposite to the gradient**

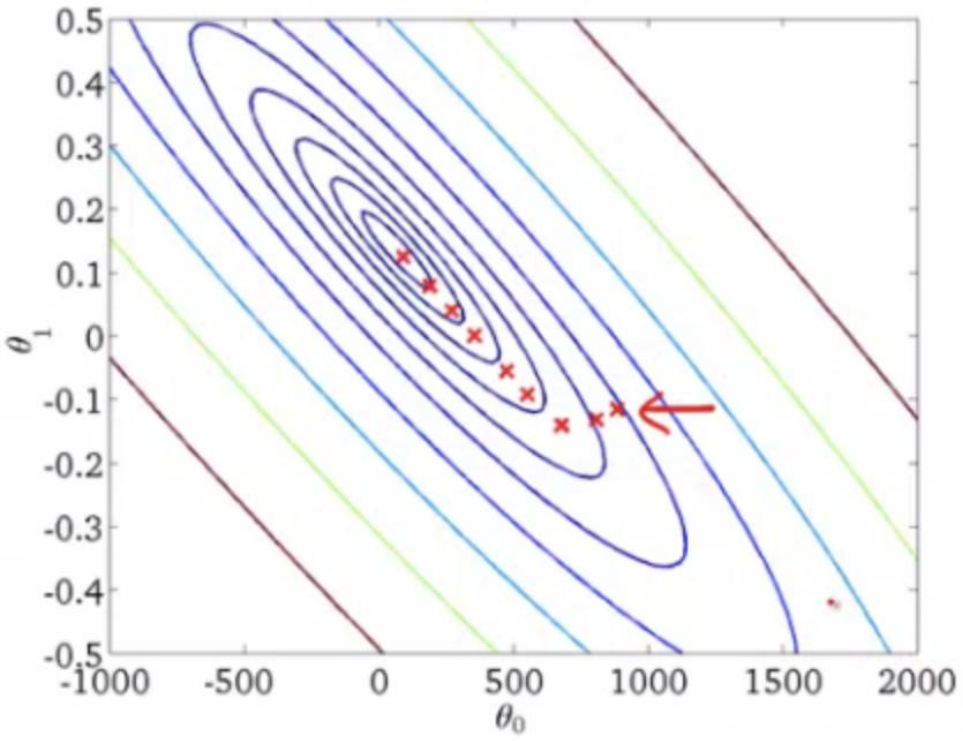
Gradient Descent Visualization

Single Parameter:
plot the dependent variable cost on the y-axis
theta on the x-axis
Two Parameter:
3-D plot :- cost on one axis and
two parameters (thetas)

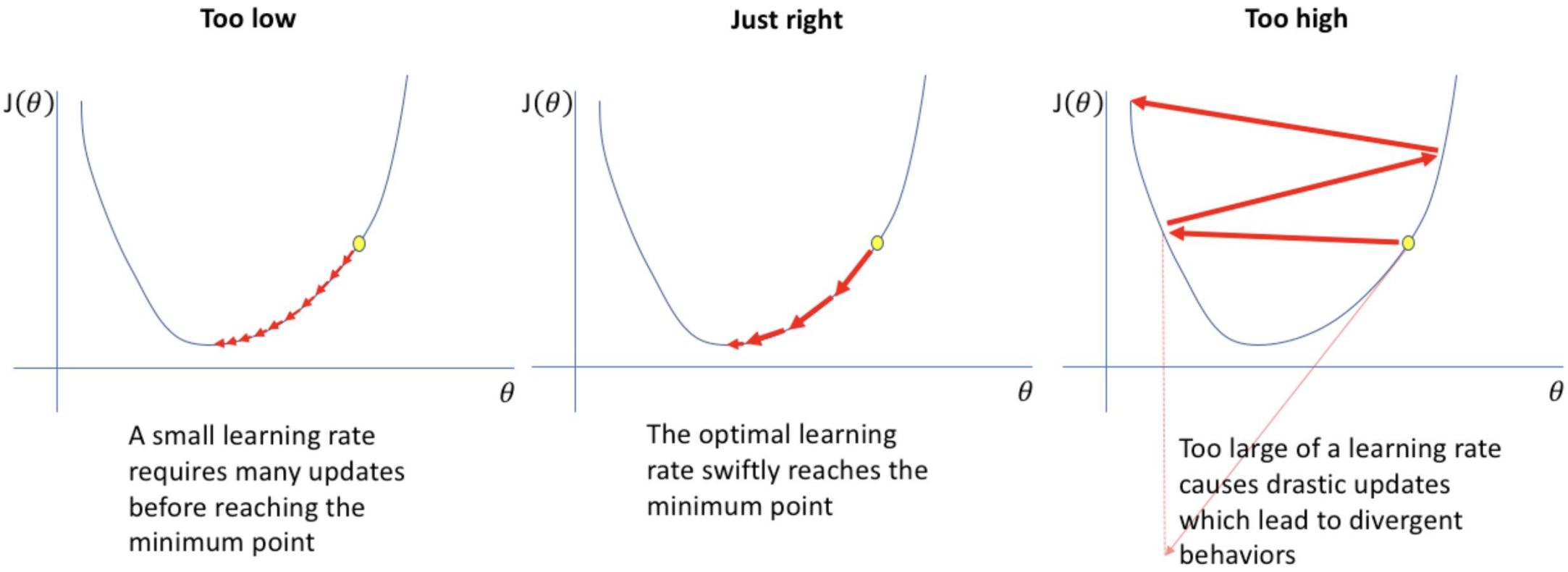


Gradient Descent Visualization

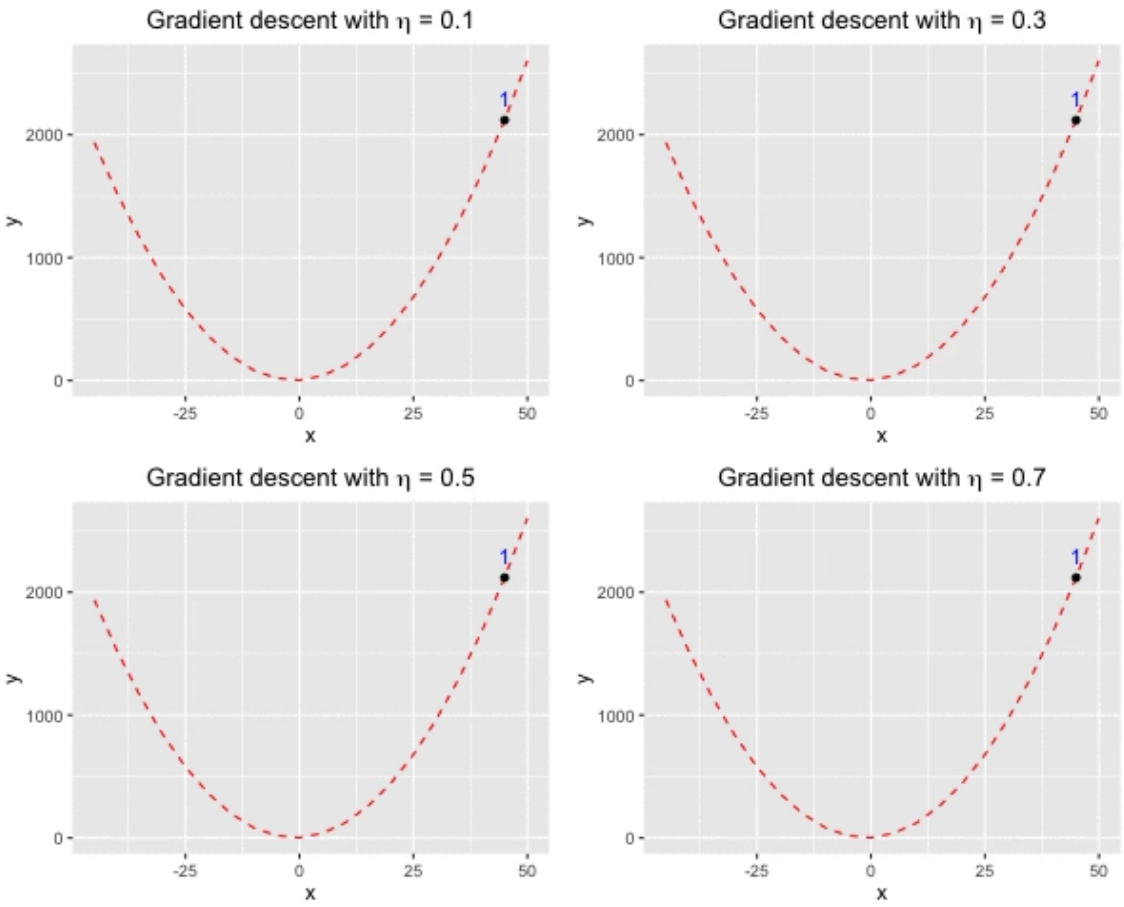
- 3-D plot in two dimensions is Contour diagram
- A contour plot is a graphical technique for representing a 3-dimensional surface by plotting constant z slices, called contours, on a 2-dimensional format
  - parameters along both axes and the response (cost) as a contour
  - value of the response increases away from the center and has same value along with the rings
  - response is directly proportional to the distance of a point from the center (along a direction)



Learning Rate

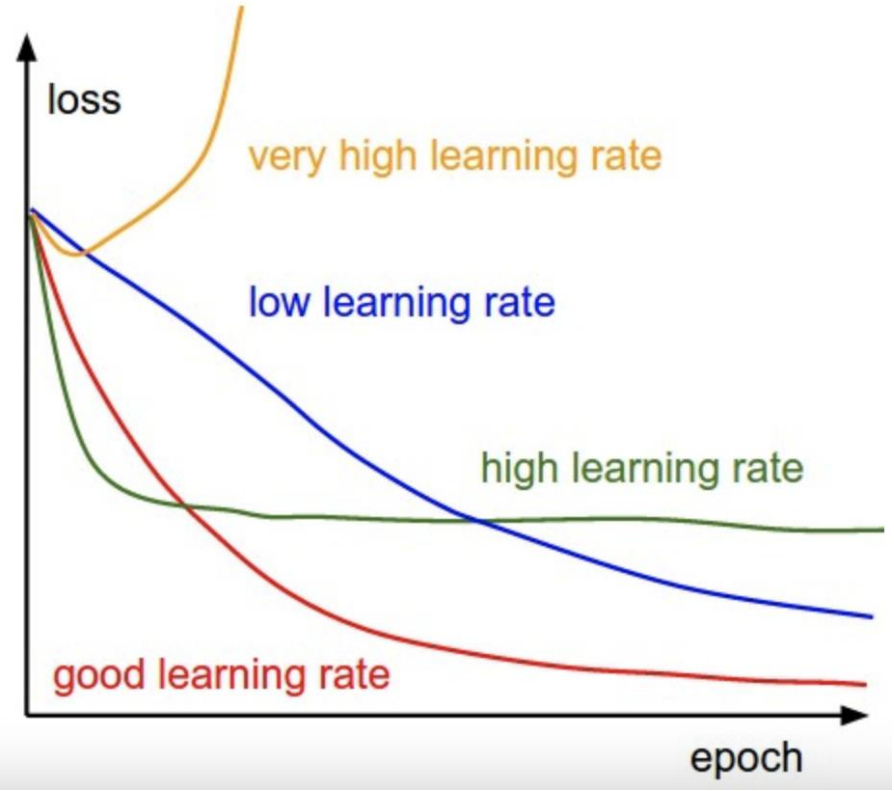


Learning Rate



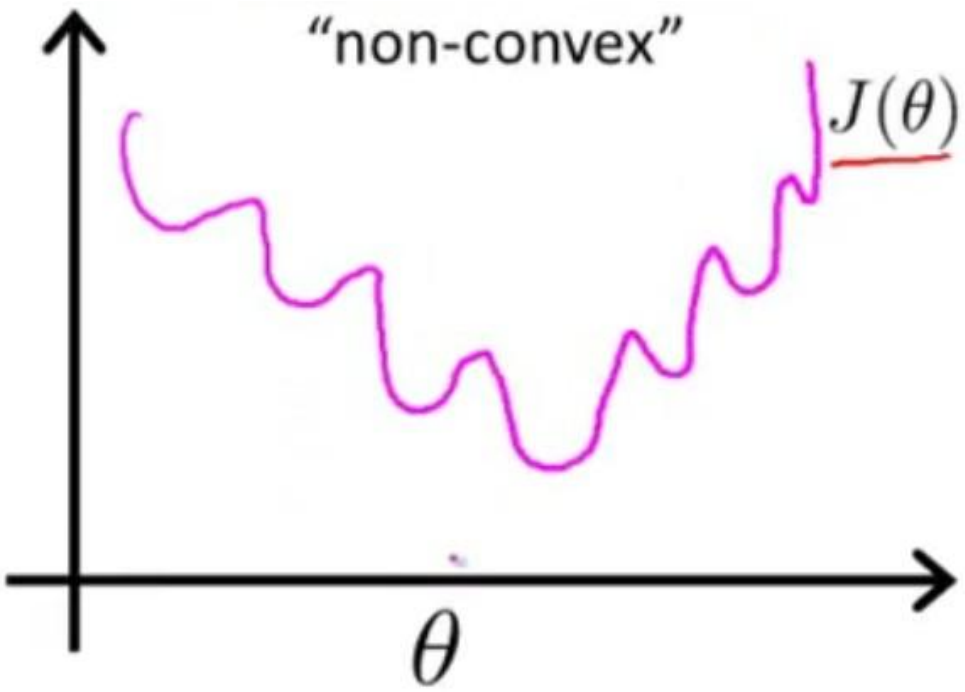
Learning Rate

- Learning rate is optimal, model converges to the minimum
- Learning rate is too small, it takes more time but converges to the minimum
- Learning rate is higher than the optimal value, it overshoots but converges
- Learning rate is very large, it overshoots and diverges, moves away from the minima, performance decreases on learning

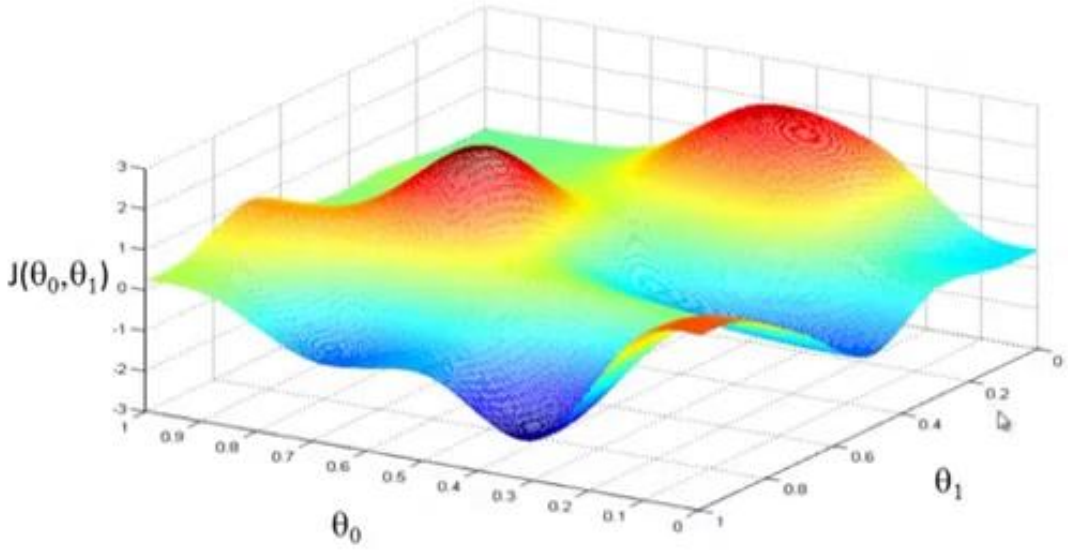


Learning Rate

- If our learning rate setting is relatively small, and we get a local optimal solution instead of the global optimal solution.



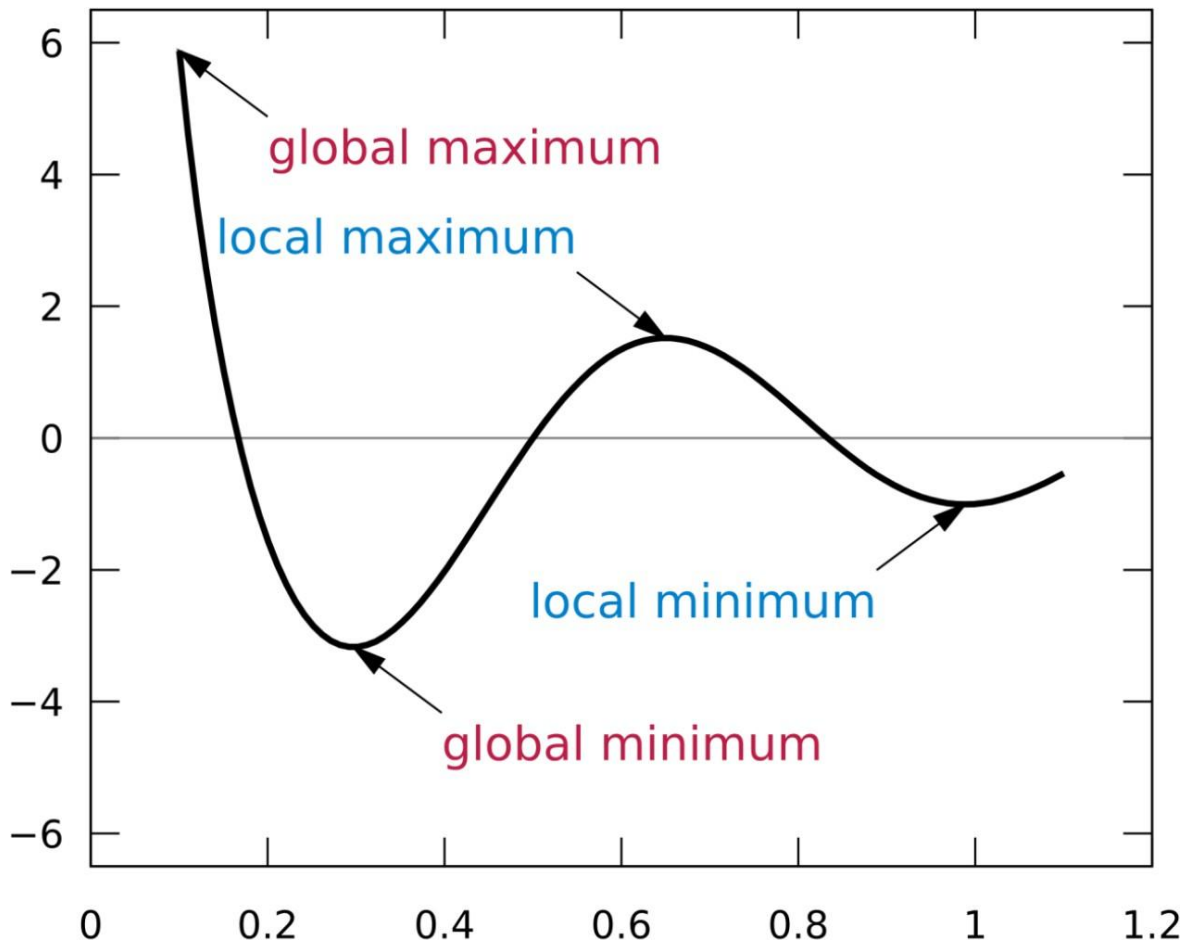
Gradient Descent for Linear Regression



Andrew Ng

Global/Local Optimization

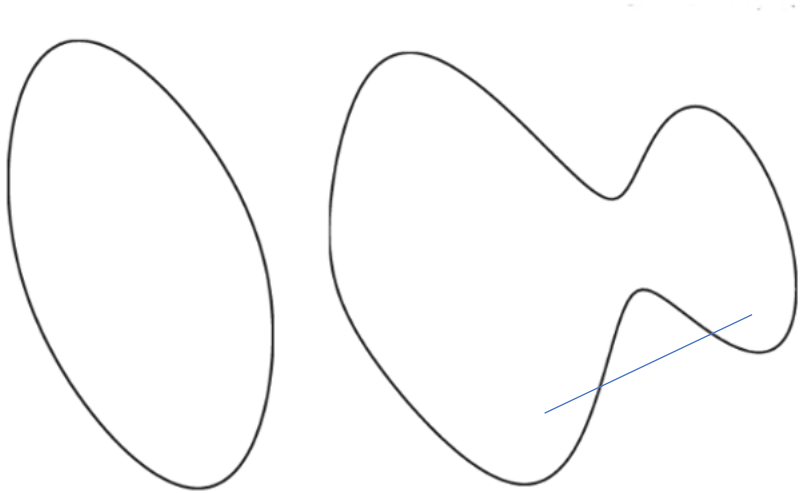
- Optimization is the process of finding the point that minimizes a function.
- A local minimum of a function is a point where the function value is smaller than or equal to the value at nearby points, but possibly greater than at a distant point.
- A global minimum is a point where the function value is smaller than or equal to the value at all other feasible points.



Convex set

- Convex set: A set C is convex if the line segment between any two points in C lies in C
- $\forall x_1, x_2 \in C, \forall \theta \in [0,1]$, then $\theta x_1 + (1-\theta) x_2 \in C$

Convex set



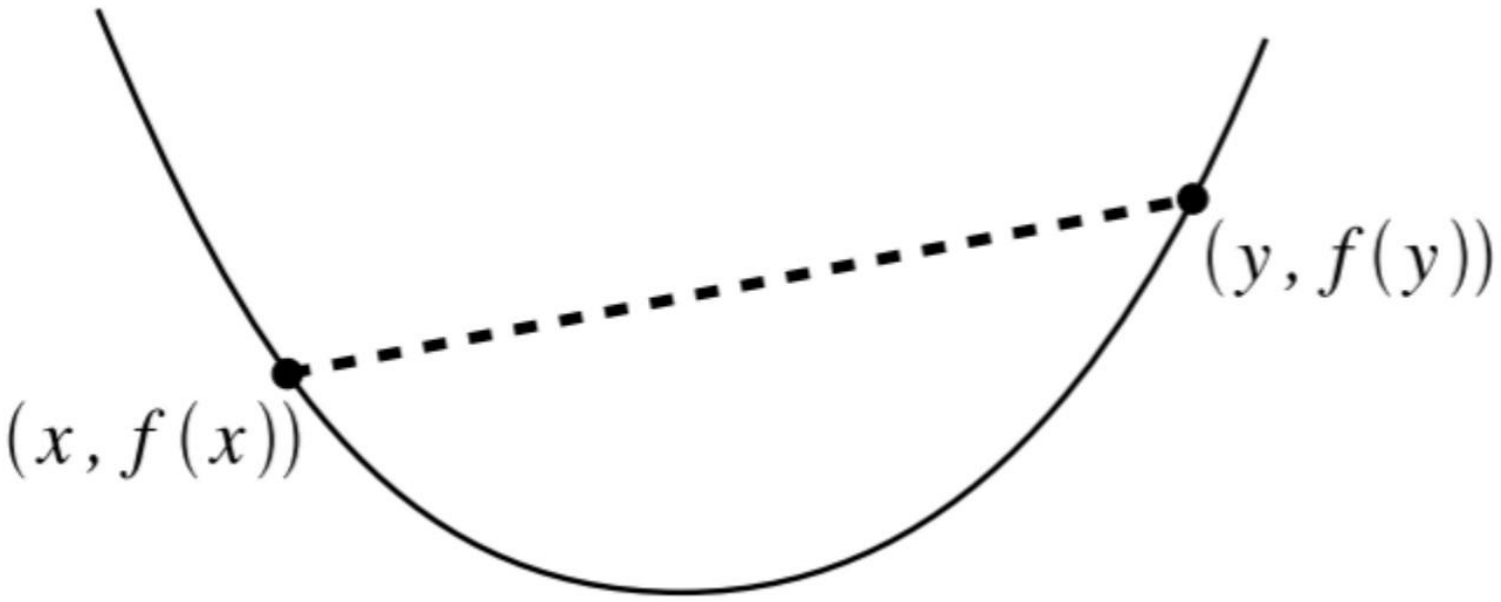
Non-convex set

Convex function

- Definition
- A function $f: \Re n \rightarrow \Re$ is convex if its domain (denoted $D(f)$) is a convex set, and if, for all $x, y \in D(f)$ and $\theta \in R$, $0 \leq \theta \leq 1$, $f(\theta x + (1 - \theta) y) \leq \theta f(x) + (1 - \theta) f(y)$.

- If we pick any two points on the graph of a convex function and draw a straight line between them, the n the portion of the function between these two points will lie below this straight line

Convex function



The line connecting two points on the graph must lie above the function

Multivariate Linear Regression

Linear Regression

- Simple linear regression
  - A single independent variable is used to predict

$$y_i = \beta_0 + \beta_1 x_i$$

- $i \in \{1, \cdots, n\}$: n observations
- x_i: independent variables
- y_i: dependent variables
- β_0: intercept (constant term)
- β_p: slope coefficients for each dependent variables
- ϵ: model's error term (residuals)

- Multiple linear regression
  - Two or more independent variables are used to predict

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip} + \epsilon$$

Multivariate linear regression

- Multiple linear regression model is

$$h_\theta(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

- If written in the form of a matrix

$$h_\theta(x) = \begin{bmatrix} \theta_0 & \theta_1 & \cdots & \theta_n \end{bmatrix} \begin{bmatrix} 1 \\ x_1 \\ \vdots \\ x_n \end{bmatrix} = \theta^T x$$

Multivariate linear regression solution

- Gradient descent: Extend the gradient formula to each parameter

$$\theta_j := \theta_j - \alpha \frac{\partial J}{\partial \theta_j} J(\theta) = (h_\theta(x_i) - y_i) x_i^{(j)}$$

  - i is the index of training sample, j is the index of features
- Least square/normal equation

$$\theta = (X^T X)^{-1} X^T y$$

Stochastic gradient descent (SGD)

Gradient Descent with more data

- $\theta_j := \theta_j - \alpha \frac{\partial J}{\partial \theta_j} \boxed{J(\theta)}$ ← All training samples
- In classic gradient descent, we compute the gradient from the loss for all training examples
- Suppose we add more examples to our training set
  - Imagine we add an extra copy of every training example
  - Gradients take 2x the time to compute
  - Costly for a single iteration
  - May not have sufficient memory
- We want to scale up to huge datasets, so how can we do this?

Stochastic gradient descent (SGD)

- Use a small set of training samples to estimate the gradient in each iteration
- $\theta_j := \theta_j - \alpha \frac{\partial J}{\partial \theta_j} \boxed{J(\theta)}$ ← A small set of training samples
- Notation:
  - Iteration: gradient update once A small set of training samples
  - Batch size: number of training samples in one iteration
  - Epoch: cycle through all training samples once

Choose a Batch size

- Batch Size = 1
  - Fast but not accurate
  - Workload is too small, hard to fully utilize computation resources
- Batch Size = # training samples
  - Slow but accurate (SGD = GD)
  - Memory issue
  - Waste computation, e.g. when all x_i are identical

Example: Problem Formulation and Solving

House Buying 101

- Pick a house, take a tour, and read facts
- Estimate its price, bid

Listing price from agent

$5,498,000	7	5	4,865 Sq. Ft.
Price	Beds	Baths	$1130 / Sq. Ft.

Redfin Estimate: $5,390,037 On Redfin: 15 days

Virtual Tour
- Branded Virtual Tour
- Virtual Tour (External Link)

Parking Information
- Garage (Minimum): 2
- Garage (Maximum): 2
- Parking Description: Attached Garage, On Street
- Garage Spaces: 2

Multi-Unit Information
- # of Stories: 2

School Information
- Elementary School: El Carmelo El
- Elementary School District: Palo A
- Middle School: Jane Lathrop Stan
- High School: Palo Alto High
- High School District: Palo Alto Un

Interior Features

Bedroom Information
- # of Bedrooms (Minimum): 7
- # of Bedrooms (Maximum): 7

- Kitchen Description: Countertop (
Dishwasher, Garbage Disposal, Ho
Island with Sink, Microwave, Over

House Buying 101

- Pick a house, take a tour, and read facts
- Estimate its price, bid

Listing price from agent

$5,498,000	7	5	4,865 Sq. Ft.
Price	Beds	Baths	$1130 / Sq. Ft.

Redfin Estimate: $5,390,037 On Redfin: 15 days

Predicted sale price

Virtual Tour
- Branded Virtual Tour
- Virtual Tour (External Link)

Parking Information
- Garage (Minimum): 2
- Garage (Maximum): 2
- Parking Description: Attached Garage, On Street
- Garage Spaces: 2

Multi-Unit Information
- # of Stories: 2

School Information
- Elementary School: El Carmelo El
- Elementary School District: Palo
- Middle School: Jane Lathrop Stan
- High School: Palo Alto High
- High School District: Palo Alto Un

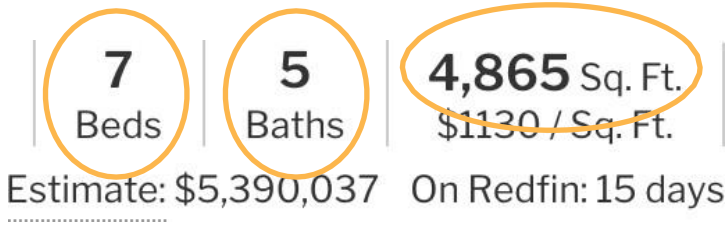
Interior Features

Bedroom Information
- # of Bedrooms (Minimum): 7
- # of Bedrooms (Maximum): 7

- Kitchen Description: Countertop Dishwasher, Garbage Disposal, H Island with Sink, Microwave, Over

A simplified model

- Assumption 1
  - The key factors impacting the prices are #Beds, #Baths, Living Sqft, denoted by x_1, x_2, x_3



- Assumption 2
  - The sale price is a weighted sum over the key factors
    - $y = w_1x_1 + w_2x_2 + w_3x_3 + b$
  - Weights and bias are determined later

Measure estimation quality (Loss/cost)

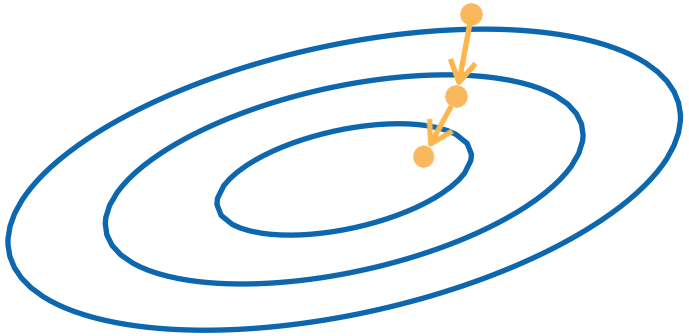
- Compare the true value vs the estimated value
- Real sale price vs estimated house price
- y: the true value
- $\hat{y}$: the estimated value
- Square loss: $l(y, \hat{y}) = (y - \hat{y})^2$

Training data

- Collect multiple data points to fit parameters Houses sold in the last 6 months
- It is called the training data/set
- The more the better
- Assume n examples

Learn Parameters

- Choose a staring point: $w^0 = [w_1, w_2, w_3, b]$
- Repeat to update the weight $t = 1,2,3$
- $w^{(t)} = w^{(t-1)} - \eta \frac{\partial l}{\partial w^{(t-1)}}$



- Gradient: a direction that increases the value
- Learning rate (η): a hyper-parameter specifies the step length

Implementation

- **Linear Regression Implementation from Scratch**
  - Linear regression
- **Concise Implementation of Linear Regression**
  - Linear regression

Visualize linear regression

- Linear regression
- Linear regression
- Linear regression